

Motor de Tarifas para Pasajes Terrestres

RETO PRÁCTICO — VERSIÓN 2

¡Felicidades por avanzar en el módulo! Ha llegado el momento de llevar nuestro motor de precios al siguiente nivel. Como desarrolladores en la plataforma chilena de venta de pasajes, su sistema original fue un éxito. Sin embargo, los clientes rara vez compran un solo pasaje a la vez.

Su nueva misión es actualizar el algoritmo central para que sea capaz de procesar **carritos de compra con múltiples pasajeros**, utilizando arreglos, métodos de iteración y aplicando estrictas buenas prácticas de codificación en JavaScript.



Objetivos a Evaluar

Con esta actividad demostrarán su dominio sobre estructuras de datos más complejas y la automatización de tareas en JavaScript. Los cuatro pilares fundamentales que se evaluarán son:

1

Arreglos y Objetos

Almacenamiento de múltiples valores y entidades en una sola variable indexada.

2

Ciclos de Repetición

Implementación de iteraciones para procesar listas de datos de manera eficiente.

3

Métodos de Arreglos

Uso de métodos funcionales inmutables (`map`, `filter`, `reduce`) para transformar y calcular datos.

4

Buenas Prácticas

Encapsulamiento en funciones reutilizables, eliminación de código redundante y control de flujo avanzado.

Fase 1 y 2: Estructuración de Datos y Control de Flujo

Fase 1 — Arreglo carritoCompras

Todo el trabajo lógico ocurre en `calculadoraTarifas.js`. En lugar de variables sueltas, se debe crear un arreglo llamado `carritoCompras` donde cada pasajero es un objeto con las propiedades: **nombre** (String), **edad** (Number), **destino** (String) y **tieneTNE** (Boolean).

El arreglo debe contener **al menos 4 pasajeros de prueba**, combinando diferentes edades, destinos y estados de pase TNE para forzar todas las validaciones de la lógica anterior.

Fase 2 — Control de Flujo en Bucles

Se debe iterar sobre `carritoCompras` usando un ciclo `for...of`. Dentro del ciclo, se valida el destino: si el destino de un pasajero **NO es "Rancagua"** (evaluado con `!==`), se usa `continue` para omitir ese pasaje y pasar al siguiente sin detener el programa.

La lógica matemática de cálculo de IVA, descuentos por tercera edad y estudiante debe encapsularse en una función reutilizable llamada `calcularPrecioFinal(pasajero)`. Los condicionales `if/else` deben refactorizarse para evitar anidaciones innecesarias.

Fase 3: Procesamiento Avanzado con Métodos de Arreglos

Una vez dominados los ciclos clásicos, se demostrará el manejo de métodos funcionales. A partir del arreglo original `carritoCompras`, se deben aplicar tres operaciones clave de manera inmutable:



.map()

Crear `boletasGeneradas`: nuevo arreglo con los objetos de pasajeros, agregando la propiedad `precioFinal` a cada uno.



.filter()

Crear `pasajerosConDescuento`: arreglo con únicamente los pasajeros que obtuvieron tarifa de tercera edad o tarifa de estudiante.



.reduce()

Aplicar sobre el arreglo transformado para calcular y acumular el **monto total** que el cliente debe pagar por todo el carrito de compras.

 Ninguno de estos métodos debe mutar el arreglo original. La inmutabilidad es un requisito estricto de evaluación.

Fase 4: Reporte Final en Consola

Utilizando el método `.forEach()` y los Template Literals aprendidos en el reto anterior, se debe generar una salida estructurada en consola que incluya dos niveles de información:

Boleta Virtual por Pasaje

Imprimir en consola una "Boleta Virtual" por cada pasaje válido procesado, interpolando las propiedades del objeto pasajero (nombre, destino, precio final, descuentos aplicados) mediante Template Literals.

Resumen Final del Carrito

Al finalizar la impresión individual, mostrar un resumen que incluya: la **cantidad total de pasajes válidos** (usando la propiedad `.length`), los **nombres de las personas que obtuvieron descuento**, y el **Total a Pagar** calculado con el método `.reduce()`.

Criterios de Calidad y Evaluación

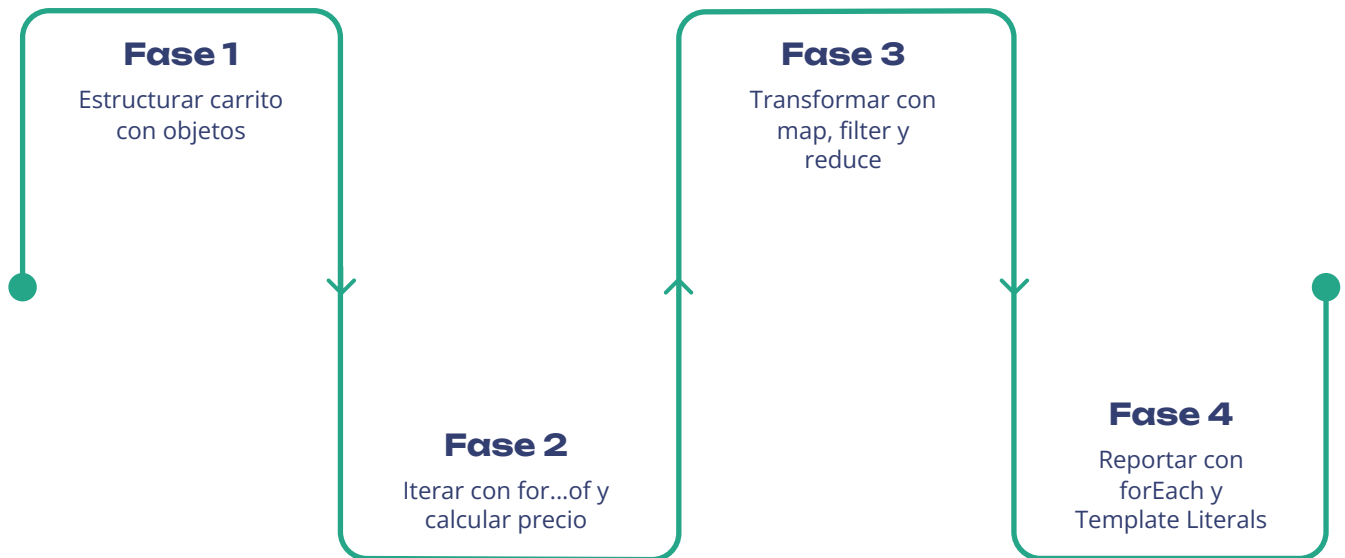
Deberán subir a la plataforma su archivo `.js` debidamente comentado. El código limpio permite que cualquier desarrollador pueda entenderlo. La evaluación se estructura en cuatro criterios principales:

Calidad del Código

Uso estricto de `const` y `let` — **cero tolerancia al uso de**

Resumen del Reto

Este reto integra todos los conceptos del módulo en un flujo de trabajo real. A continuación, el mapa completo de las fases y entregables que componen la solución esperada:



Al completar las cuatro fases, habrán construido un motor de tarifas robusto, modular y profesional, capaz de procesar múltiples pasajeros, aplicar descuentos automáticamente y generar reportes detallados en consola.

- ✓ Recuerden: el entregable es el archivo `calculadoraTarifas.js` debidamente comentado, subido a la plataforma antes de la fecha límite.